

MANUAL TÉCNICO

1. Información General

Lenguaje de Programación

Python 3

Tipo de Proyecto

Aplicación de consola para administración y generación de imágenes utilizando estructuras de datos.

2. Introducción

El presente proyecto consiste en una aplicación desarrollada en Python que permite gestionar capas, usuarios e imágenes utilizando estructuras de datos dinámicas.

El sistema implementa Árboles Binarios de Búsqueda (ABB), listas circulares dobles y generación de imágenes mediante pixeles.

Además, el proyecto permite visualizar estructuras mediante Graphviz y generar imágenes PNG utilizando la librería Pillow.

3. Objetivos

Objetivo General

Desarrollar una aplicación que implemente estructuras de datos para la administración y generación de imágenes por capas.

Objetivos Específicos

- Implementar árboles binarios de búsqueda.
- Implementar listas circulares dobles.
- Realizar carga masiva de datos.
- Generar imágenes utilizando pixeles.
- Implementar recorridos ABB.
- Generar reportes gráficos.
- Administrar usuarios e imágenes.

4. Requisitos del Sistema

Requisitos de Hardware

- Procesador Intel o AMD
- 4 GB RAM mínimo
- 500 MB de almacenamiento disponible

Requisitos de Software

- Python 3.10 o superior
- Visual Studio Code
- Graphviz instalado

5. Librerías Utilizadas

Pillow

Utilizada para:

- Crear imágenes
- Dibujar pixeles
- Guardar imágenes PNG

Importación:

```
from PIL import Image, ImageDraw
```

Graphviz

Utilizada para:

- Graficar árboles
- Visualizar conexiones

Importación:

```
from graphviz import Digraph
```

6. Instalación del Proyecto

Paso 1 – Crear entorno virtual

```
python3 -m venv .venv
```

Paso 2 – Activar entorno virtual

Windows

```
.venv\Scripts\activate
```

Linux/Mac

```
source .venv/bin/activate
```

Paso 3 – Instalar dependencias

```
pip install pillow  
pip install graphviz
```

7. Organización del Proyecto

El sistema fue desarrollado utilizando una arquitectura modular.

7.1 Carpeta estructuras

Contiene las estructuras de datos.

Archivo	Descripción
abb_capas.py	ABB de capas
abb_usuarios.py	ABB de usuarios
lista_imagenes.py	Lista circular doble

7.2 Carpeta modelos

Contiene modelos principales.

Archivo	Descripción
capa.py	Modelo de capa
pixel.py	Modelo de pixel
usuario.py	Modelo de usuario

7.3 Carpeta cargas

Contiene carga masiva de datos.

Archivo	Descripción
carga_capas.py	Carga capas
carga_imagenes.py	Carga imágenes
carga_usuarios.py	Carga usuarios

7.4 Carpeta generacion

Contiene generación de imágenes.

Archivo	Descripción
generador_imagen.py	Generación PNG

7.5 Carpeta reportes

Contiene reportes gráficos.

Archivo	Descripción
graphviz_reportes.py	Árbol capas
grafo_usuarios.py	Árbol usuarios
grafo_imagen.py	Grafo imagen

8. Arquitectura del Sistema

El sistema se divide en módulos independientes.

Cada módulo posee una responsabilidad específica.

Esto permite:

- reutilización de código
- mantenimiento sencillo
- mejor organización
- escalabilidad

9. Estructuras de Datos Utilizadas

9.1 Árbol Binario de Búsqueda (ABB)

Utilizado para almacenar:

- capas
- usuarios

Cada nodo posee:

- hijo izquierdo
- hijo derecho

Inserción ABB

La inserción se realiza comparando IDs o nombres.

Ejemplo

```
    3
   / \
  1   5
   \ /
   2 4
```

9.2 Lista Circular Doble

Utilizada para almacenar imágenes.

Cada nodo contiene:

- siguiente
- anterior

Representación

[A] <-> [B] <-> [C]
^ |
|_____|

10. Modelos Implementados

10.1 Modelo Pixel

Representa un pixel individual.

Atributos

Atributo	Descripción
----------	-------------

x	Coordenada X
y	Coordenada Y
color	Color hexadecimal

Ejemplo

```
Pixel(  
    100,  
    120,  
    "#FFFF00"  
)
```

10.2 Modelo Capa

Representa una capa de la imagen.

Atributos

Atributo	Descripción
----------	-------------

id	Identificador
----	---------------

pixeles	Lista de pixeles
---------	------------------

10.3 Modelo Usuario

Representa usuarios del sistema.

Atributos

Atributo	Descripción
----------	-------------

nombre	Nombre usuario
--------	----------------

imagenes	Lista imágenes
----------	----------------

11. Carga Masiva de Datos

El sistema implementa carga masiva mediante lectura de archivos.

Tipos de Archivos

Extensión	Uso
-----------	-----

.cap	capas
------	-------

.usr	usuarios
------	----------

.im	imágenes
-----	----------

Ejemplo Archivo Usuarios

maria

carlos

luis
alejandro

Ejemplo Archivo Capas

1
2
3
4
5

12. Generación de Imágenes

La generación de imágenes se realiza utilizando Pillow.

Proceso

1. Crear imagen vacía
2. Recorrer ABB
3. Dibujar pixeles
4. Guardar PNG

Fondo utilizado

"#87CEEB"

13. Recorridos Implementados

13.1 Inorden

Izquierda → Raíz → Derecha

13.2 Preorden

Raíz → Izquierda → Derecha

13.3 Postorden

Izquierda → Derecha → Raíz

14. Administración de Usuarios

El sistema permite:

- agregar usuarios
- eliminar usuarios
- buscar usuarios
- agregar imágenes
- eliminar imágenes

15. Reportes Generados

El sistema genera automáticamente:

Reporte	Descripción
arbol_capas.png	ABB capas
arbol_usuarios.png	ABB usuarios
grafo_imagen.png	Relaciones capas
imagen_general.png	Imagen completa
imagen_inorden.png	Recorrido inorden
imagen_preorden.png	Recorrido preorden
imagen_postorden.png	Recorrido postorden

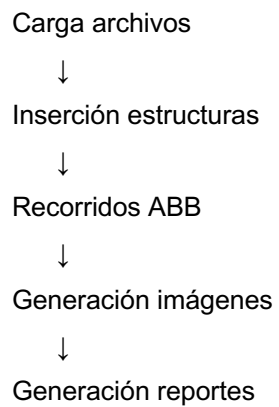
16. Menú Principal

La aplicación funciona mediante menú interactivo.

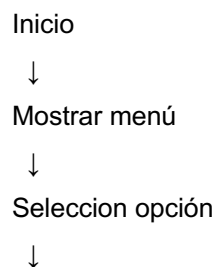
Funcionalidades

- Cargar capas
- Cargar imágenes
- Cargar usuarios
- Generar imágenes
- Mostrar recorridos
- Administrar usuarios
- Generar reportes

17. Diagrama General del Sistema



18. Flujo del Programa



Ejecutar función



Mostrar resultado



Regresar menú

19. Manejo de Errores

El sistema contempla validaciones para:

- archivos inexistentes
- usuarios inexistentes
- imágenes inexistentes
- capas inexistentes
- entradas inválidas

20. Complejidad Computacional

Inserción ABB

Complejidad promedio:

$O(\log n)$

Búsqueda ABB

Complejidad promedio:

$O(\log n)$

Recorridos ABB

Complejidad:

$O(n)$

21. Ventajas del Sistema

- Organización modular
- Fácil mantenimiento
- Escalable
- Visualización gráfica
- Aplicación de estructuras dinámicas

22. Conclusiones

El proyecto permitió implementar estructuras de datos aplicadas a un entorno gráfico.

Se utilizaron:

- árboles binarios
- listas dobles
- manejo de archivos
- programación orientada a objetos
- generación gráfica

Además, se logró integrar múltiples módulos trabajando en conjunto para generar imágenes dinámicas y reportes visuales.